

NCE 播放器工程学习指南

从时间标签到点读事件链

核心结论 0.3 秒偏移会提前标签；真实方法链已验证，真实浏览器播放仍需另验

这次学习什么

只研究播放器工程：LRC 解析、时间索引、句界、播放控制与高亮。使用原创短句，不重制教材或课程。

实际验证了什么

2026-10-05 实测 14 个原始上游模块。DOM、音频、存储和时钟为替身。本次公开整理未重跑；代码与 MIT 许可完整保留。

还没有验证什么

真实浏览器、音频与教学效果未测。HTML 浏览器及转 PDF 原路线受阻，本次未重试。HTML 仅结构检查；PDF 直接生成并逐页检查。

从浅到深的十二个问题

- | | |
|-------------------|-------------------|
| 01 这个项目到底是什么 | 02 资源如何变成播放器状态 |
| 03 一行 LRC 怎么变成句子 | 04 0.3 秒偏移朝哪边 |
| 05 播放到某一刻该亮哪一句 | 06 一句的终点从哪里来 |
| 07 点击和播放更新如何接起来 | 08 点读和循环怎样不同 |
| 09 播放速度和计时器是一回事吗 | 10 为什么要节流 防抖和进度恢复 |
| 11 它与词锚定语义时间有什么关系 | 12 测试全绿能证明什么 |

01 这个项目到底是什么

把媒体秒数变成可点选的句子

NCE 的工程核心是一个静态网页播放器。静态是指界面与逻辑由 HTML、CSS 和 JavaScript 文件提供；音频、时间标签和文字来自另外的资源目录。它把“现在播到第几秒”转换成“现在应该高亮哪一句”。

想象给自己录的一段演讲做遥控目录：点某句话，播放器跳到那一秒；继续播放时，界面跟着换行。这里学习的是这种导航机制，不是四册教材内容，也不评价课文、音频或翻译质量。



同一套播放逻辑可以接不同的合法资源；文字与录音的使用权需要分别确认。

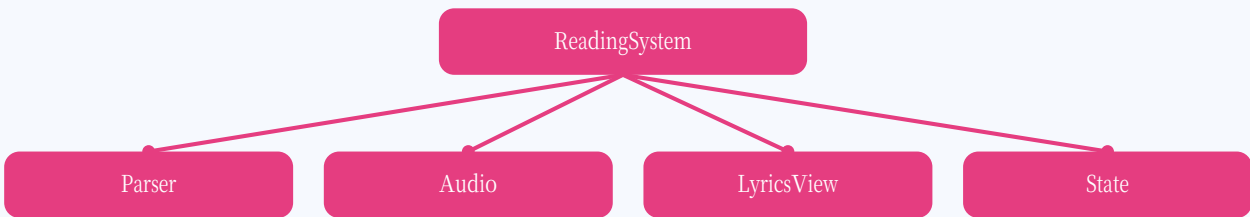
源码 README.md ReadingSystem.js

02 资源如何变成播放器状态

协调器负责接线 各模块各做一件事

模块就是承担一类职责的代码单元。ReadingSystem 是协调器：BookService 读取目录，PrefetchService 取资源，LRCParse 解析句子，AudioController 控制媒体，LyricsView 显示和高亮文字。状态是程序此刻记住的信息，例如当前单元、当前句子和循环模式。

本练习把 DOM、音频对象、存储和时钟换成可控制的替身。保留上游模块字节不变；只在实验子类中覆盖 init()，阻止自动拉取课本目录，再注入原创句子。构造器的真实接线和后续方法仍会执行。



替身替换运行环境；并没有重新写一份解析算法冒充上游实现。

源码 ReadingSystem.js AudioController.js LyricsView.js

时间标签的含义

03 一行 LRC 怎么变成句子

先匹配时间标签 再拆分双语文字

LRC 是把时间标签写在文本行开头的一种格式。本解析器接受 [mm:ss.xx] 或 [mm:ss.xxx]：分钟为一到两位，秒为两位，小数为两到三位；秒数达到 60 的行被跳过。注释、空行、格式不匹配和英文为空的行也被跳过。

例如自写 [00:01.20]Check the lamp. | 检查灯。时间先变成 1.2 秒，再减偏移量；竖线左侧进入 english，右侧进入 chinese。若出现第三段竖线文本，它仍留在 fullText，却不会成为第二段翻译。这是当前代码的具体边界，并非完整 LRC 标准解析器。



时间、英文和中文是结构化字段；最终按时间升序排列，不能只按输入行号理解。

源码 LRCParse.js

04 0.3 秒偏移朝哪边

解析器执行 标签时间减去偏移量

时间偏移是让字幕整体提前或延后显示的校正量。当前 CONFIG 的 TIME_OFFSET 为 0.3；标签 1.2 秒被解析为 0.9 秒，因此正偏移会提前。结果按毫秒精度四舍五入。

这里最容易踩坑的是开头：标签 0.1 秒减 0.3 后得到 -0.2 秒，解析器保留这个负值。在媒体时长有限且为正时，AudioController.seek 把请求夹在 0 与时长之间，因此 -0.2 会写入 0。未知或零时长时，它以请求值作上限；替身中负值仍可被写入，不能概括为总会归零。



只有已知且为正的时长分支才保证这里的负定位变成 0；真实媒体对未知时长的行为未测。

源码 config.js LRCParse.js AudioController.js

05 播放到某一刻该亮哪一句

从后往前找最后一个不晚于当前时刻的句子

时间查询采用倒序遍历：找到 `currentTime` 大于或等于句子时间的第一项就返回索引。索引是数组里的位置，从 0 开始。第一句开始前返回 -1；正好达到某个时间标签时，该句已经生效。

如果两个句子时间相同，排序后靠后的那句会被命中。像站牌上同时出现两个出发项，当前规则选最后一项，并不替你消除歧义。重复时间要由数据检查或新的产品规则处理；不能从绿色测试推断“字幕数据没有问题”。



精确命中使用 `>=`；重复时间的最后一项获胜，之前的项可能没有独立可见时段。

源码 `LRCParser.js`

06 一句的终点从哪里来

通常借下一句的起点作为当前句的终点

句界是句子的开始与结束时间。`getSentenceBoundaries` 以当前句时间为起点，以下一句时间为终点。最后一句没有下一句，就取音频时长与“起点加 0.1 秒”中的较大值。非数组输入，或索引小于 0、达到数组长度时返回 `null`；它没有全面校验索引类型。

因此重复时间会让前一句出现 `startTime` 等于 `endTime` 的零长度范围。最后一句的兜底也不是语音识别得到的真实结尾，只是程序保证至少一个很短区间的办法。要判断停顿是否自然，仍需要合法录音和真实播放验证。



标签提供的是起点；句尾由下一项或兜底推导，不是对声音内容的理解。

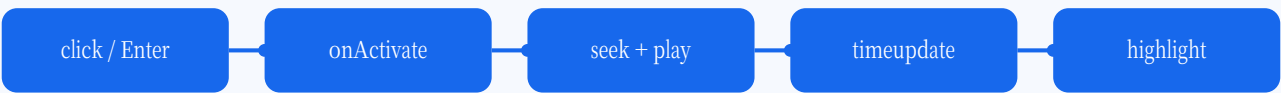
源码 `LRCParser.js ReadingSystem.js`

07 点击和播放更新如何接起来

事件是信号 回调把信号接到方法

事件是在某个动作发生时发出的信号；回调是收到信号后调用的函数。LyricsView 用事件委托，在共同容器上监听句子点击和 Enter 或空格，再读出 data-index 与 data-time。ReadingSystem 接着设置高亮、seek、play，并保存进度。

反方向也有一条链：音频发出 timeupdate，AudioController 调用 onTick，ReadingSystem 处理句界并同步高亮，LyricsView 修改 active 类。本实验发送替身事件，让真实方法链执行，并检查类、时间和存储记录；没有把“调用了回调”写成“人耳听到了播放”。



点句子是主动定位；timeupdate 是播放过程反馈。两个方向必须分别验证。

源码 LyricsView.js AudioController.js ReadingSystem.js

08 点读和循环怎样不同

同一句界 不同模式决定到点之后做什么

状态机是“当前状态加输入决定下一步”的规则集合。这里 loopMode 有 off、click、one、list、book 五种取值。click 和 one 都会锁住选中的句子；达到终点时，二者都 seek 回句首，但 click 会暂停，one 则继续。

list 使用媒体对象自身的 loop；book 在 ended 事件后切到下一个单元。它们不是同一个开关。实验主要验证 click 与 one 的句界行为及真实事件链；跨课本网络加载和真实媒体循环，不在本次组件替身验证范围内。



“点读完成”会回到句首并停下；它并不是停留在句尾。

源码 config.js ReadingSystem.js

09 播放速度和计时器是一回事吗

媒体坐标和现实等待时间要分开看

媒体时间是录音内部的位置，例如第 8 秒；墙钟时间是现实世界过去了多久。playbackRate 是播放速率，改变单位现实时间内走过多少媒体时间。它不会自动重写 LRC 标签的媒体坐标。

在理想连续播放且无停顿时，一段 4 秒媒体用 2 倍速约 2 秒播完，但句子仍在原来的媒体秒数开始。本实验控制时钟以稳定触发节流，并观察播放速率属性；没有解码音频，也没有验证真实音高、响度、延迟或主观听感。



倍速是一种坐标推进速率；重新录音或剪辑则可能改变句子与秒数的对应关系。

源码 AudioController.js config.js helpers.js

10 为什么要节流 防抖和进度恢复

高频刷新与低频保存需要不同节奏

节流限制调用频率。AudioController 的 timeupdate 处理器按约 1000/30 毫秒的最小间隔执行；这不是浏览器保证每秒 30 帧的承诺。防抖则等待调用安静下来再做事，ReadingSystem 将常规进度保存延后 1500 毫秒；暂停、页面隐藏等路径另有即时保存。

恢复进度还要等 loadedmetadata，媒体时长可用后才定位。待恢复时间太大时，当前实现最多放到 duration 减 0.05 秒。替身存储证明键值写入逻辑；它不证明浏览器隐私模式、跨设备同步或真实本地存储配额都正常。



节流让刷新别太密，防抖让保存别太勤；两者都不能替代媒体加载完成事件。

源码 helpers.js storage.js AudioController.js ReadingSystem.js

11 它与词锚定语义时间有什么关系

一个锚定词的身份 一个使用已有媒体秒数

词锚定语义时间先指向“哪一个词”，再通过对齐过程获得实际音频时间。NCE 的 LRC 则直接携带媒体秒数。两者可以在一条流水线上连接，但概念不能合并成同义词：稳定的词标识和当前录音的第几秒解决的是不同问题。

已发布 Hypit 笔记里的相关实验是静态源码分析加固定语速模拟，不是已经完成的真实音频对齐。可复用的是区分时间坐标与证据层级的思路。本次知识关联据实际读到的笔记提出，不能据此认定读者已熟练掌握。复用时先核对同义条目，再增加具体证据。



桥梁是“词到实际音频时间”的映射；更换录音后，原来的映射可能失效。

源码 LRCParse.js

12 测试全绿能证明什么

证明被覆盖的规则 没有覆盖的体验仍需验证

单元测试检查局部函数，组件集成测试检查多个真实模块怎样接线；端到端测试则应从真实用户界面、真实资源到最终输出走完整流程。本练习包含前两类。DOM、音频、存储和时钟是受控替身，所以不能称为真实浏览器端到端通过。

可重复的证据是：固定代码版本、逐字节校验、原创输入、完整命令、断言结果和执行时间。下一步若要上线，需要另外确认资源授权、真实浏览器、网络加载、音频解码和播放策略。MIT 代码许可不自动授予教材、音频或翻译内容的再分发权。



本包跑通前两层；后两层是下一次有资源和环境时才能完成的验收。

源码 LICENSE README.md ReadingSystem.js

获得的能力和可以做的事

01 解析双语时间行

从受支持的 LRC 行提取时间和文字，过滤无效行并排序

02 按媒体时刻定位句子

使用倒序含等号查询，并推导句子的起止边界

03 把点选动作接到媒体控制

句子点击或键盘激活触发定位、播放与高亮

04 管理几种播放状态

区分点读、单句循环、媒体循环和单元轮播的职责

05 保存并恢复播放进度

通过存储键、元数据事件与延迟保存协调进度

四个具体用途

1. 为自己录制或已获授权的演讲制作逐句导航，先用原创小样例验证数据规则
2. 为时间标签写离线检查器，提前发现重复起点、负时间或无效格式
3. 把替身测试方法迁移到其他媒体组件，快速回归点击、进度和循环逻辑
4. 设计从词级对齐结果到 LRC 的转换工具，明确词身份与媒体秒数之间的映射

如何复用已有知识

历史公开知识范围：394 条索引元数据、19 篇实际全文阅读的公开笔记；其余 375 篇未读正文。先核对同义条目，再补充案例与证据；不以笔记存在代替掌握或真实产品验收。

动手实验 从解包到时间边界

将练习 ZIP 放到当前目录；第 2 至 6 步继续在已解包目录执行

01 解包与确认源码完整

目的 在全新目录运行，确认环境及 14 个模块和许可的字节。

```
python3 -m zipfile -e nce-player-exercise.zip replay
cd replay/nce-player-exercise
python3 --version
node --version
node run.mjs --group integrity --output results/01-integrity.json
```

实测 17/17：15 个文件哈希、14 个 JS 数量及无外联保护通过。

验证 核对 verifiedUpstreamFiles=15, failed=0。

2026-10-05 09:59:34 UTC | 实测耗时 0.158 秒

02 读出六条原创双语句子

目的 观察过滤规则、双语字段与按时间排序的实际结果。

```
node run.mjs --group parser --output results/02-parser.json
```

实测 7/7：六条原创记录按时间排序；无效格式被过滤。

验证 查看 tests 中六项 parser 行的 actual 与 expected。

2026-10-05 09:59:34 UTC | 实测耗时 0.099 秒

03 对照偏移与句子边界

目的 让 0 与 0.3 秒偏移的差别成为可见的数值。

```
node run.mjs --group timeline --output results/03-timeline.json
```

实测 11/11：首句 -0.2 秒；2.2 秒命中索引 3；索引 2 为零长度。

验证 对照两组时间数组、精确命中和最后一句 [5.7,8]。

2026-10-05 09:59:34 UTC | 实测耗时 0.101 秒

动手实验 从媒体事件到组件接线

04 检查媒体控制事件

目的 在时长、进度条与时钟替身上执行真实 AudioController。

```
node run.mjs --group controller --output results/04-controller.json
```

实测 15/15：已知 8 秒时长下定位受限；恢复进度为 7.95 秒。

验证 未知时长负定位是特征记录；tick、拖动和销毁均有断言。

2026-10-05 09:59:34 UTC | 实测耗时 0.090 秒

05 检查句子显示与激活

目的 调用真实 LyricsView，观察渲染字符串、active 类与事件。

```
node run.mjs --group view --output results/05-view.json
```

实测 11/11：六个句子节点、高亮、点击与键盘激活通过。

验证 检查转义字符串、旧高亮移除和空列表；不是浏览器布局验收。

2026-10-05 09:59:34 UTC | 实测耗时 0.109 秒

06 贯通真实组件方法链

目的 验证点击到播放，以及 tick 到高亮和句末暂停的接线。

```
node run.mjs --group integration --output results/06-integration.json  
node run.mjs --output results/07-all.json
```

实测 16/16；再跑完整集合 72/72。click 句尾回到 0.95 秒并暂停。

验证 全量为 55 项行为、16 项完整性和 1 项保护；非真实端到端。

2026-10-05 09:59:35 UTC | 实测耗时 0.206 秒

结果怎么读与下一次验收

看得见的中间成果

在 timeline 结果里对照零偏移与 0.3 秒偏移，观察负时间、精确命中与重复时间。controller 与 integration 结果进一步展示 seek、暂停、循环和高亮记录。这些是程序状态的可见变化。

本次边界

全部测试通过只覆盖本包断言。替身 DOM 不等于浏览器布局或可访问性验收；替身音频不等于自动播放策略、解码与实际扬声器；受控时钟不等于真实定时精度。不要把组件集成写成真实端到端。

后续验收清单

拿到自己拥有或获授权的录音后，在真实浏览器检查加载、CORS、播放策略、点击与键盘操作、字幕提前或滞后、重复时间、句末停顿以及刷新恢复。重新剪辑音频后重新核对时间标签。

固定源码与核对位置

[项目说明与资源分离 README.md](#)

[时间格式 查询 句界 js/utils/LRCParser.js](#)

[媒体定位 事件 进度恢复 js/player/AudioController.js](#)

[渲染 激活 高亮 js/ui/LyricsView.js](#)

[协调 循环 进度保存 js/ReadingSystem.js](#)

[偏移与模式配置 js/config.js](#)

[MIT 许可 LICENSE](#)

[知识关联 Hypit 已发布项目笔记](#)

[知识关联 词锚定语义时间 已发布概念笔记](#)

源码链接固定到上述提交。ZIP 保留 MIT 全文、14 个模块与七份历史 JSON；本次未重跑。软件许可不覆盖教材、翻译或音频的再分发权。笔记与日志保留关联、命令和结果。